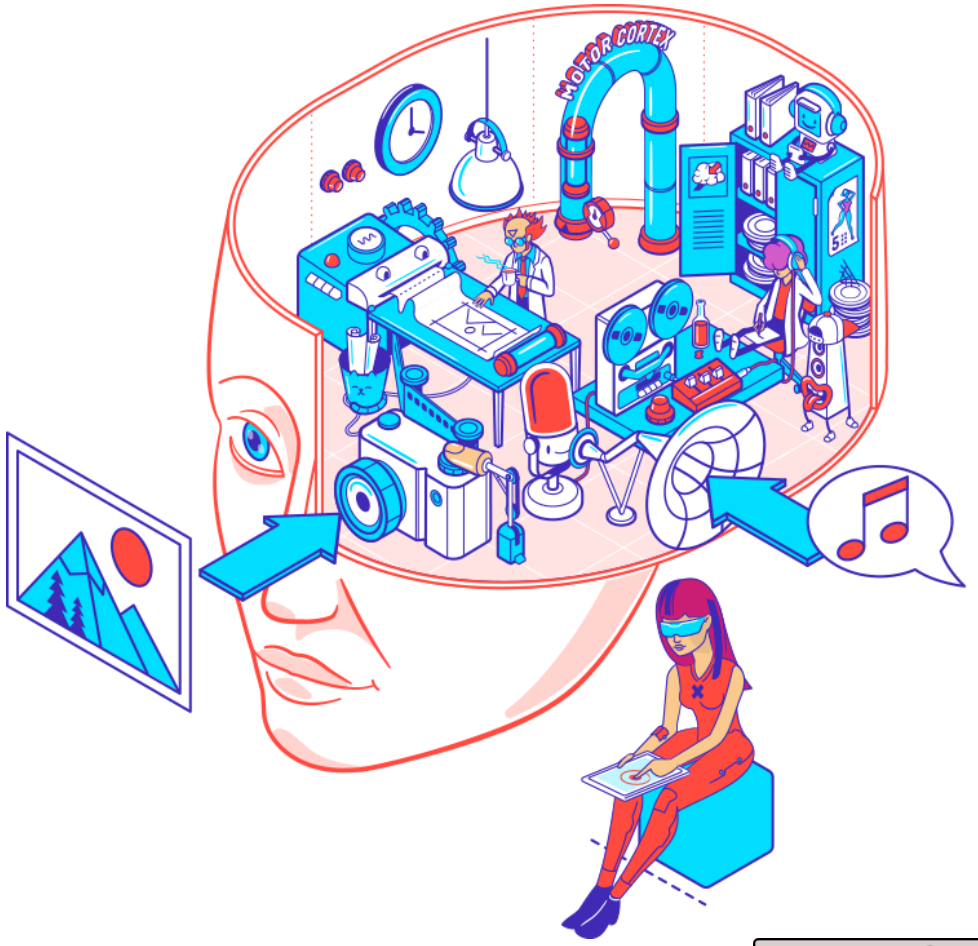
HAW HAMBURG

MEDIA PROGRAMMING

UNIT 7: ANIMATED GRAPHICS



Prof. Dr. Eike Langbehn

Department of Media Technology
Faculty of Design, Media and Information
Hamburg University of Applied Sciences

CC BY

AGENDA:

1. Introduction
2. From Java to Python
3. Raster Graphics
4. Algorithmic Graphics Generation
5. Vector Graphics
6. Interactive Graphics
7. **Animated Graphics**

LEARNING OUTCOME

- Understanding **technical foundations** of animations
- Knowledge about **different approaches** of animating graphics

BASICS

UNIT 7: ANIMATED GRAPHICS



INTRODUCTION

THE ILLUSION OF MOTION

Animation is not "moving" objects, but a series of static images shown in rapid succession. This works due to **Persistence of Vision**—the human eye's ability to retain an image for a fraction of a second.

TECHNICAL BASICS

Frames: The individual static images that make up the sequence.

Framerate (FPS): The number of frames displayed per second. Higher rates result in smoother motion.

ART VS. TECHNOLOGY

While animation is first and foremost an **artistic medium**—defined by timing, appeal, and storytelling—this unit focuses on the **technical foundations** that make these digital illusions possible.

THE ILLUSION OF MOVEMENT

?

KEY QUESTION

HOW DO YOU CREATE THE ILLUSION OF MOVEMENT IN DIGITAL GRAPHICS?

Previous Foundations

- → **Unit 3: Raster Graphics:** Pixel-based Images (used as Sprites)
- → **Unit 4: Algorithmic Graphics Generation:** Mathematical Functions
- → **Unit 5: Vector Graphics:** Transformations, Rendering
- → **Unit 6: Interactive Graphics:** Main Loop and Delta Time

CONCEPTS

FRAME-BY-FRAME

The classic approach: every single frame is manually defined (e.g., Sprite Sheets). Total control but high effort.

KEYFRAMES

Define state at specific points in time. The computer interpolates (tweens) the values in between.

PROCEDURAL

Movement is generated mathematically (e.g., Sine waves, Noise, Physics engines) in real-time.

STATE-BASED

Transitions between discrete animation states (Idle, Run, Jump) controlled by logic and triggers.

CONTEXT: REALTIME VS. OFFLINE

Realtime

Calculated instantly (games, UI). Must maintain high FPS; limited complexity per frame.

Offline

Pre-rendered (movies, clips). Frames can take hours to calculate; unlimited visual complexity.

FRAME-BY-FRAME

UNIT 7: ANIMATED GRAPHICS



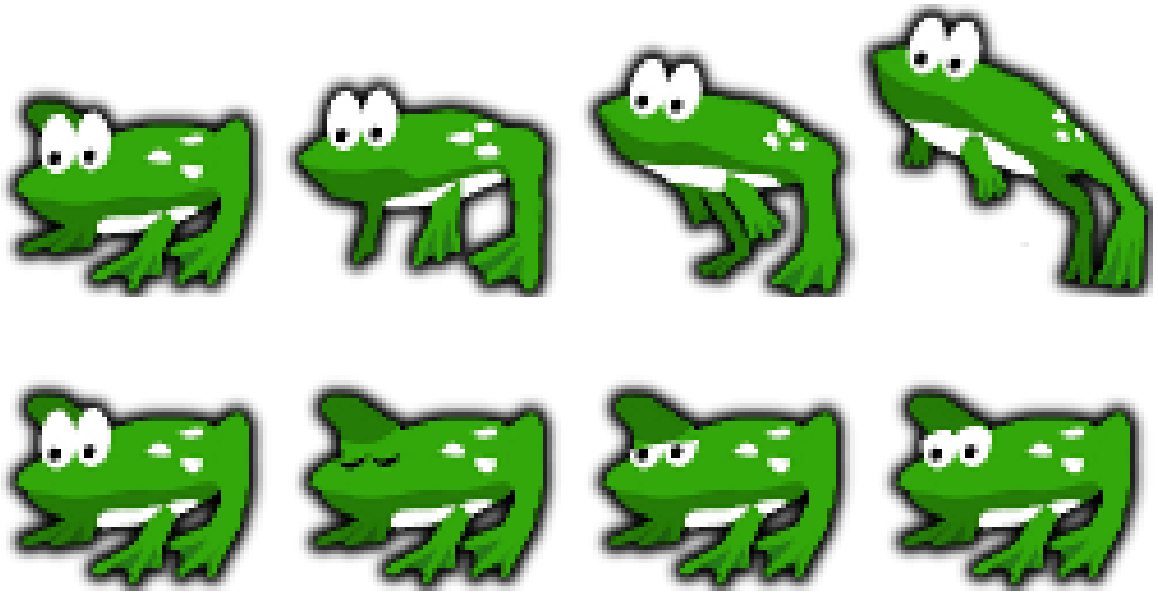
WHAT IS FRAME-BY-FRAME?

The most fundamental type of animation. Each frame is a unique image, played back in a specific order to create the illusion of movement.

- High artistic control
- Basis for classic 2D pixel art
- Handled via **Sprite Sheets** or individual files



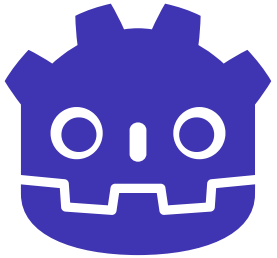
THE SPRITE SHEET CONCEPT



Efficiency is key. Instead of many small files, we pack frames into a single **Sprite Sheet** texture.

- Uses UV coordinates to "pick" the right frame.
- Reduces draw calls for the GPU.
- Managed in Godot by **SpriteFrames** resources.

GODOT: ANIMATEDSPRITE2D



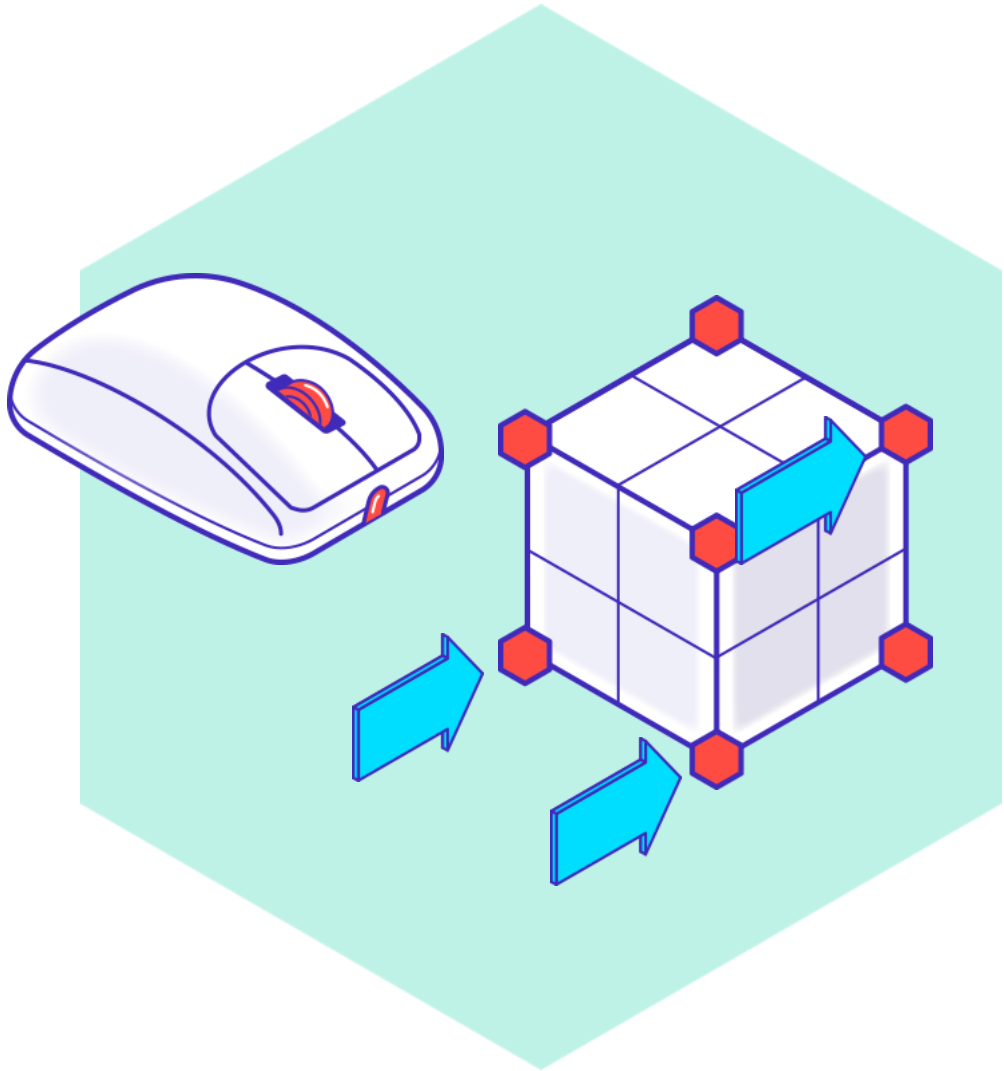
1. Add an **AnimatedSprite2D** node to your scene.
2. In the Inspector, create a new **SpriteFrames** resource.
3. Use the **Animations Panel** to add animations (e.g., "walk", "idle").
4. Click "Add frames from Sprite Sheet" and slice your texture.
5. Toggle **Autoplay** or control via script:

```
$AnimatedSprite2D.play("walk")
```

More Information on 2D Sprite Animations in the Godot documentation

KEY-FRAME ANIMATIONS

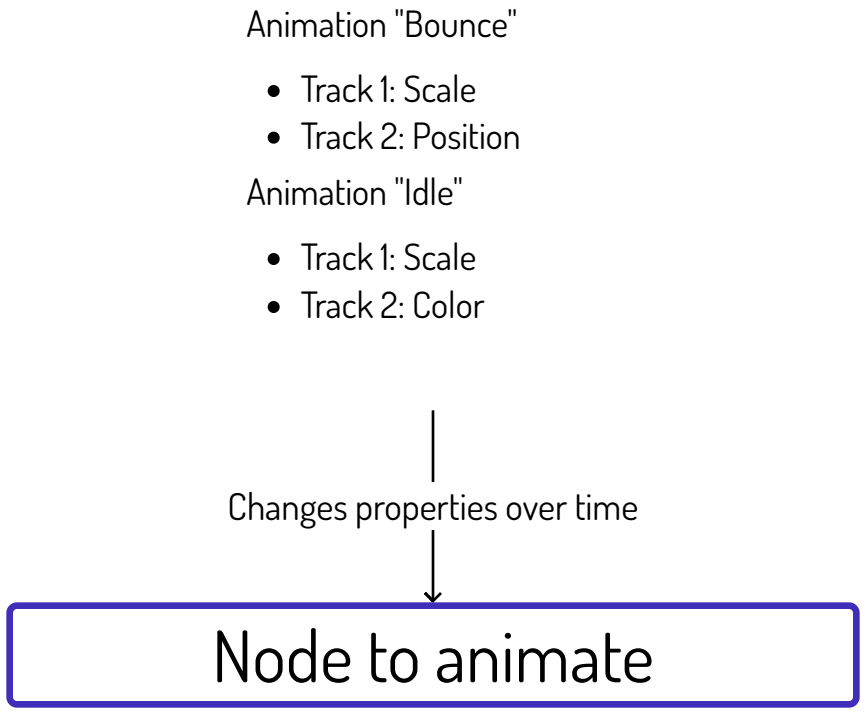
UNIT 7: ANIMATED GRAPHICS



THE ANIMATION SYSTEM

- Animation works by **changing properties** of nodes **over time**: e.g. position, color, character pose
- `AnimationPlayer` nodes play animations
- Each animation contains tracks, with each track defining how a single property should change over time

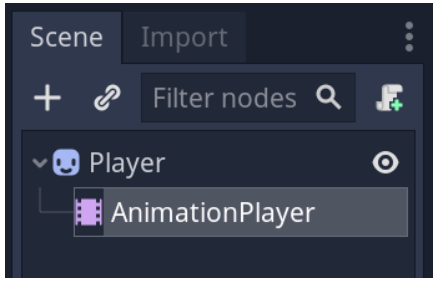
`AnimationPlayer` Node

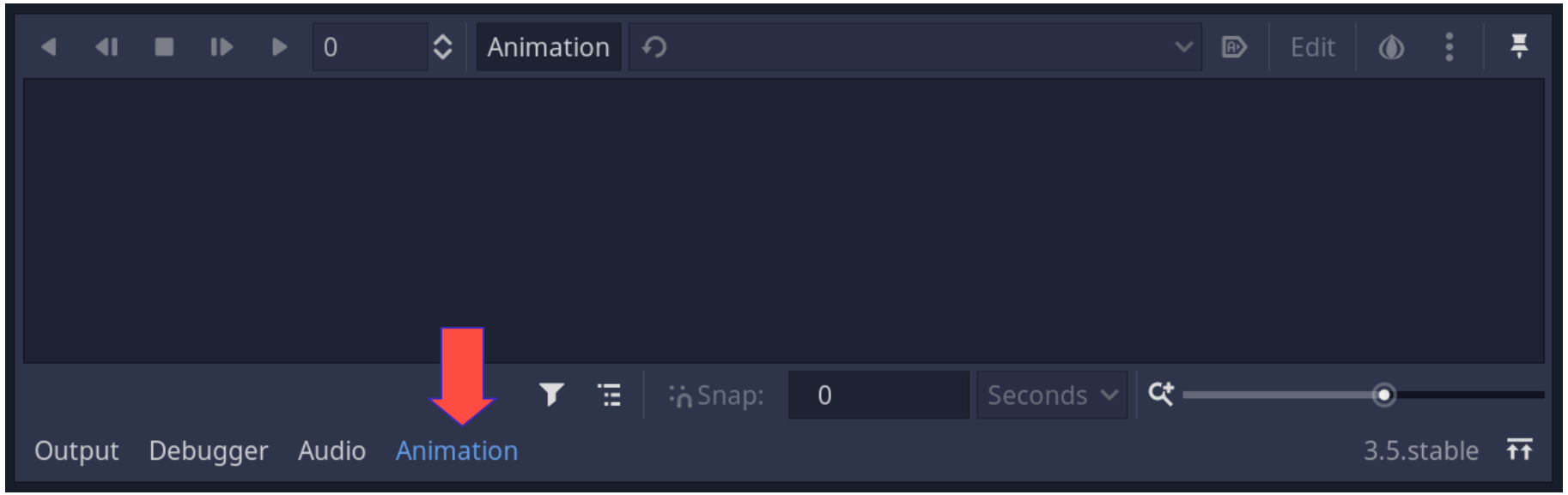


More Information on Animations in the Godot documentation

ANIMATIONPLAYER SETUP

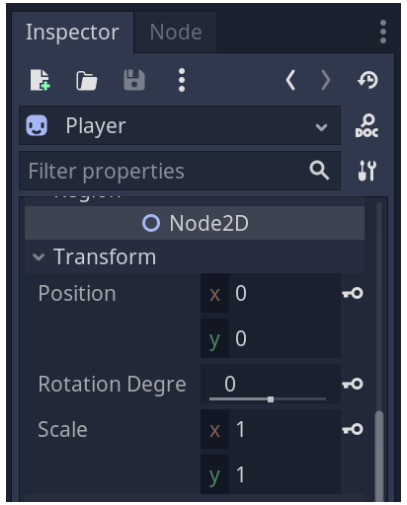
- Create an `AnimationPlayer` Node anywhere in the scene
- Open the Animation tab at the bottom of the editor window

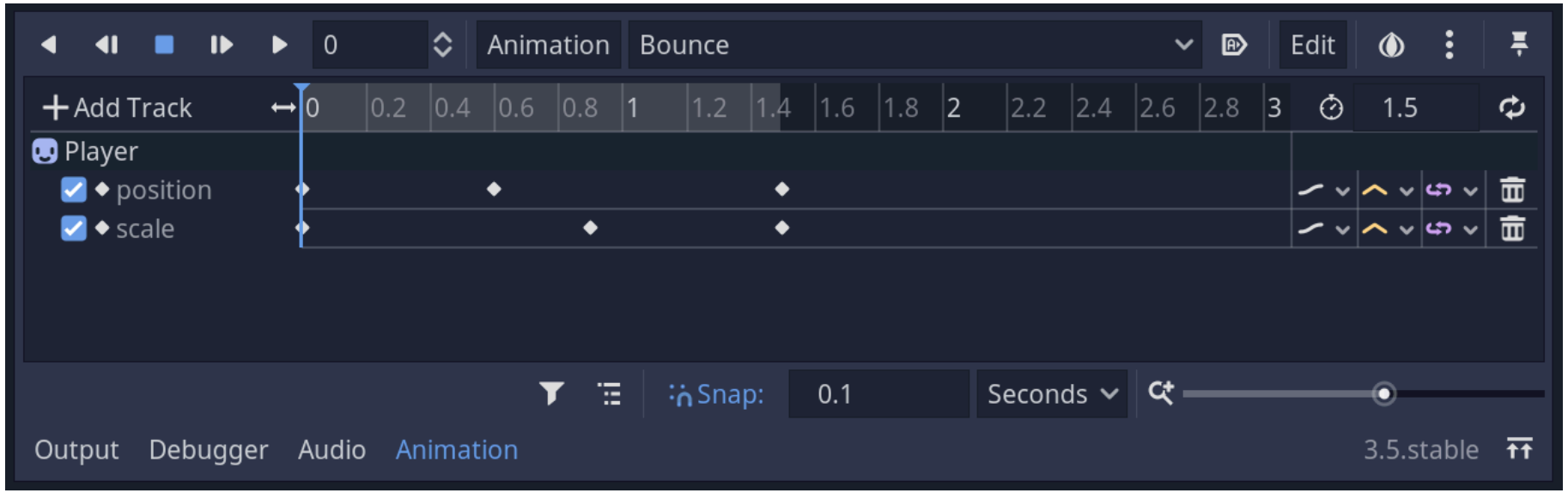




CREATING AN ANIMATION

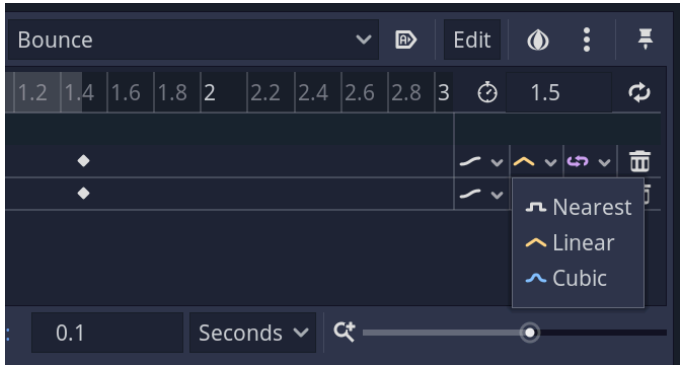
- Create a new animation
- Set the animation length
- Select the node to animate in the scene tree
- Create keyframes





ANIMATION DETAILS

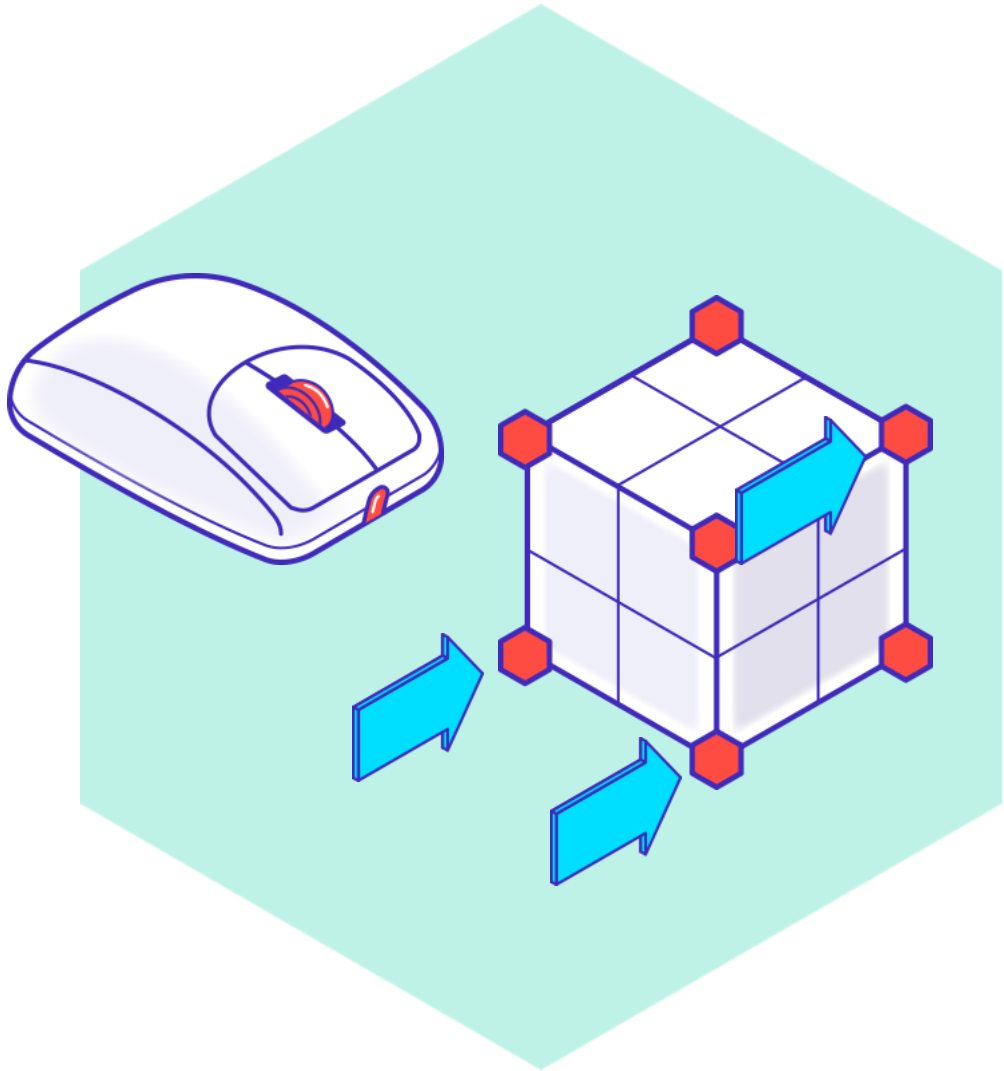
- Values between keyframes are **interpolated**
- Selecting a fitting interpolation function can improve animation appearance
- Animations can be configured to loop
- When playback stops, the property values will reflect the last keyframes of each track, both in the editor and in the game, which can be a nuisance
- To fix this, each `AnimationPlayer` has a "reset" animation containing a "default pose"
- When saving a scene, properties are reset to the first frame of this animation





PROCEDURAL ANIMATIONS

UNIT 7: ANIMATED GRAPHICS



INTERPOLATION & EASING

WHAT IS TWEENING?

Short for **"in-betweening"**. It is the process of generating intermediate frames between two keyframes to create the appearance of smooth motion.

Linear Interpolation (lerp) is a common way of finding a value between two points:

```
result = a + (b - a) * t
```

What is *t*? The interpolation factor (0.0 to 1.0):

t = 0.0: We are exactly at point **A**

t = 0.5: We are exactly in the **middle**

t = 1.0: We are exactly at point **B**

Easing Functions modify the weight *t* to create more natural movement:

- **Ease-In:** Starts slow, then accelerates.
- **Ease-Out:** Starts fast, then decelerates.

Common Easing Formulas:

Quadratic In: $f(t) = t^2$

Quadratic Out: $f(t) = 1 - (1 - t)^2$

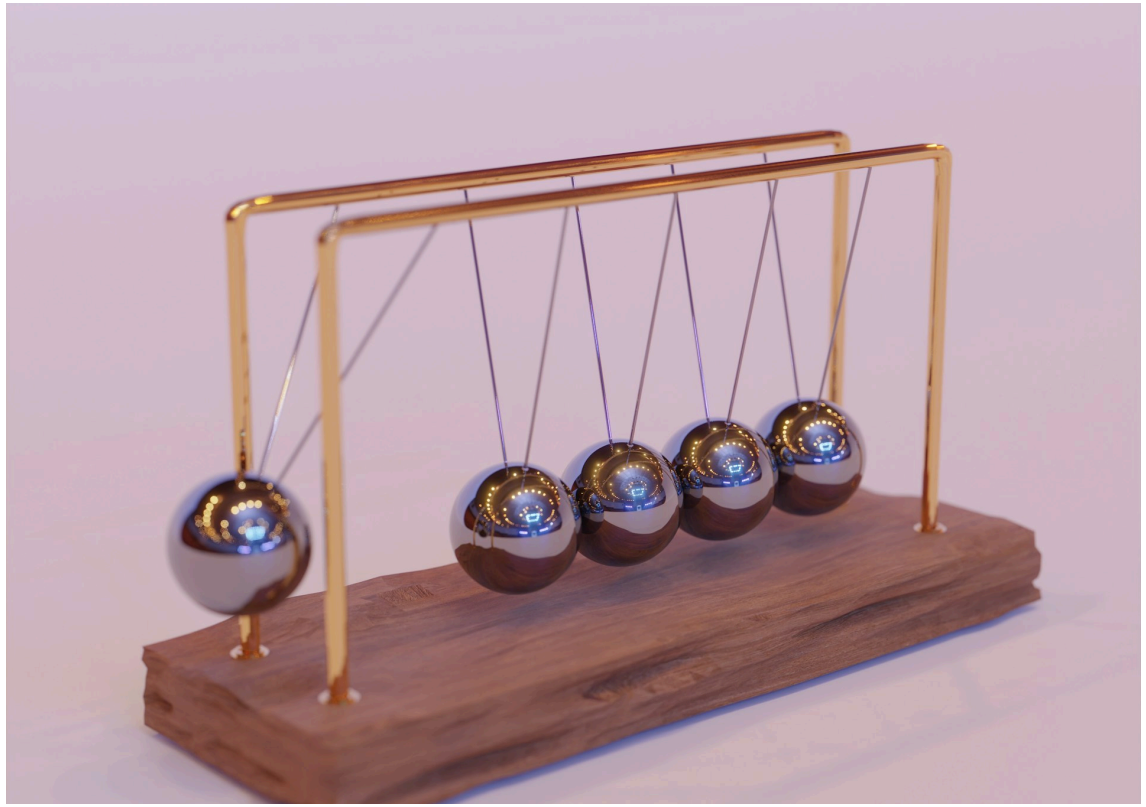
Smoothstep: $f(t) = 3t^2 - 2t^3$

OSCILLATION & CIRCULAR MOTION

Using **Sine** and **Cosine** functions allows for repetitive, smooth procedural movement.

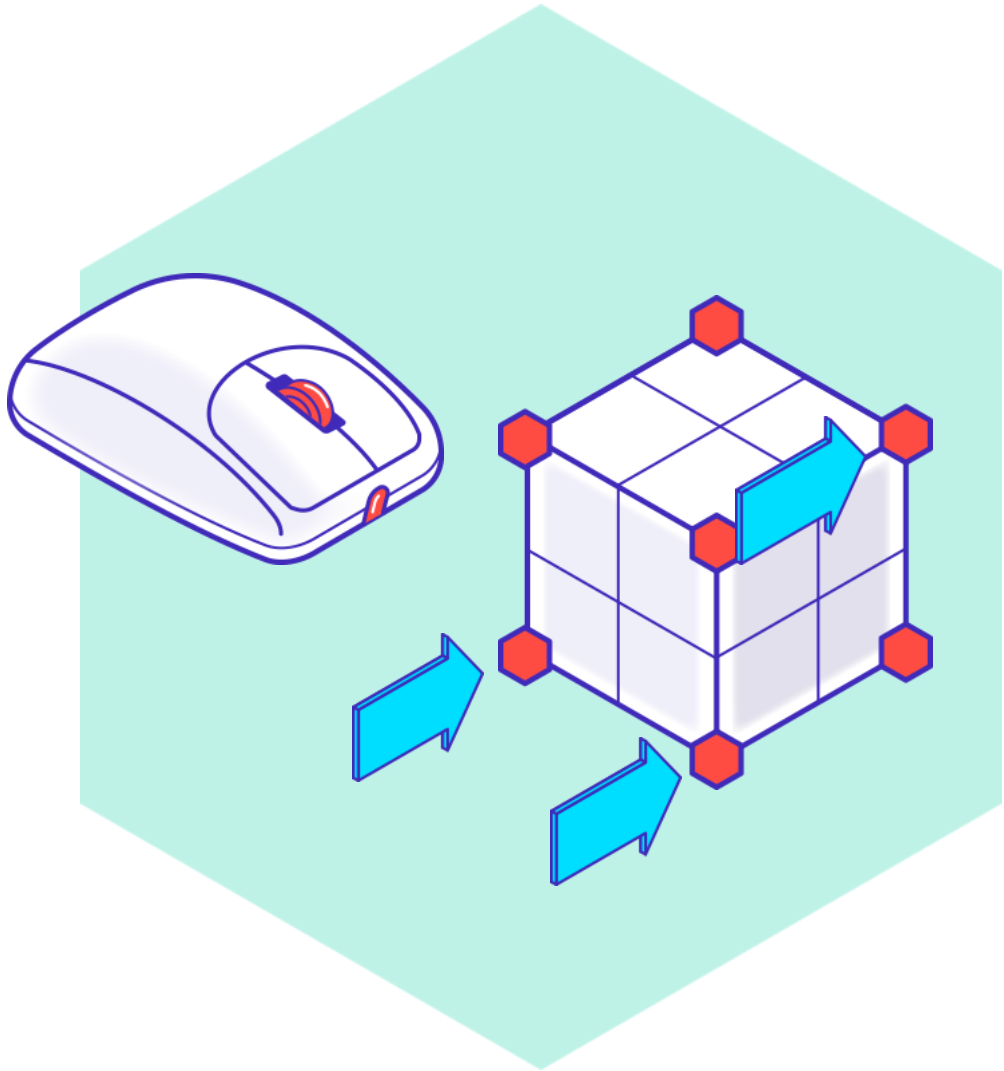
```
// Pendulum (Sine)
x = sin(time * speed) *
amplitude
```

```
// Circular Motion
x = cos(angle) * radius
y = sin(angle) * radius
```



STATE-BASED ANIMATIONS

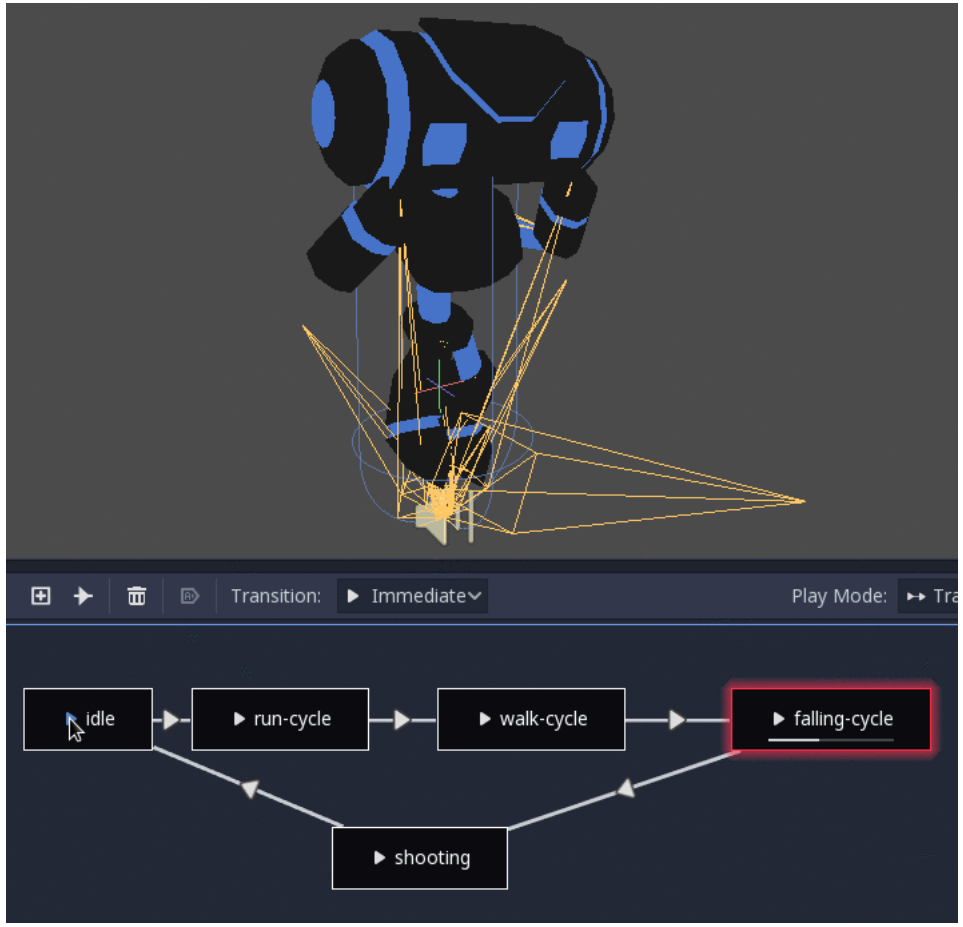
UNIT 7: ANIMATED GRAPHICS



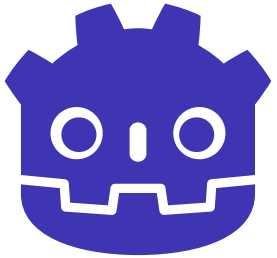
ANIMATION STATE MACHINES

A **Finite State Machine (FSM)** organizes complex animations by breaking them into discrete chunks.

- **States:** Individual animations (Idle, Walk, Jump).
- **Transitions:** The paths between states.
- **Triggers/Conditions:** Logic that decides when to switch (e.g., `is_on_floor`). Transition triggers are usually based on variables like `velocity` or `input_direction`.



GODOT: ANIMATIONTREE



1. Add an **AnimationTree** node to your scene.
2. Set **Tree Root** to `AnimationNodeStateMachine`.
3. Assign your **AnimationPlayer** to the node.
4. In the editor, right-click to add nodes (Idle, Run, etc.).
5. Connect them with arrows to create transitions.
6. Control via code:

```
$AnimationTree["parameters/playback"].tra
vel("Jump")
```

More Information on Animation Tree in the Godot documentation

HAW HAMBURG

Unless otherwise noted, the contents of this Open Educational Resource are licensed under the Creative-Commons Attribution 4.0 International license (CC BY 4.0)
Attribution: Rafael Epplée, Eike Langbehn, Peter Wood

